Cost, Latency & Error Handling

Token Economics & Production Reliability

Day 4 - Session 2

KavinSchool

Token Economics

Understanding What You're Paying For

KavinSchool

How LLM Pricing Works

You pay per token, not per request. Input and output tokens are priced differently:

Model	Input (per 1M tokens)	Output (per 1M tokens)
GPT-4o	$2.50	$10.00
GPT-4o-mini	$0.15	$0.60
Claude 3.5 Sonnet	$3.00	$15.00
Gemini 2.5 Pro	$1.25	$10.00
Gemini 2.5 Flash	$0.15	$0.60

> Key insight: Output tokens cost 3-5x more than input tokens. Shorter responses save real money.

Cost Estimation in Practice

```python
import tiktoken

def estimate_cost(messages, response, model="gpt-4o"):
    """Estimate the cost of an API call."""
    enc = tiktoken.encoding_for_model(model)

    # Count input tokens
    input_tokens = sum(len(enc.encode(m["content"])) for m in messages)
    input_tokens += len(messages) * 4  # per-message overhead

    # Count output tokens
    output_tokens = len(enc.encode(response))

    # GPT-4o pricing
    COST_PER_1M = {"input": 2.50, "output": 10.00}
    input_cost = (input_tokens / 1_000_000) * COST_PER_1M["input"]
    output_cost = (output_tokens / 1_000_000) * COST_PER_1M["output"]

    return {
        "input_tokens": input_tokens,
        "output_tokens": output_tokens,
        "total_cost": input_cost + output_cost,
    }
```

Real-World Cost Scenarios

Use Case	Tokens/Request	Requests/Day	Monthly Cost (GPT-4o)
Simple Q&A	~500	100	~$1.90
RAG chatbot	~2,000	500	~$37.50
Code review	~4,000	200	~$30.00
Doc summarization	~8,000	50	~$15.00

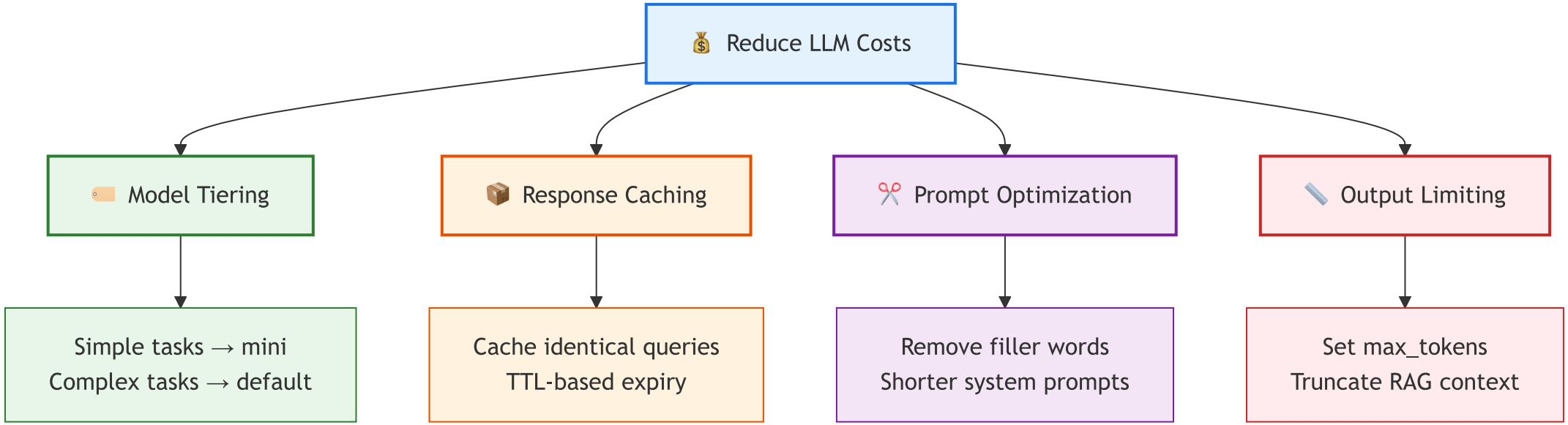
> Same workloads with GPT-4o-mini: Divide costs by ~15x.

Cost Optimization

Spending Less Without Losing Quality

KavinSchool

Cost Optimization Strategies



- **Model Tiering** - Route simple tasks to cheaper, faster models (e.g., mini).
- **Response Caching** - Reuse exact matches using TTL-based expiry.
- **Prompt Optimization** - Keep system prompts concise and remove filler words.
- **Output Limiting** - Set `max_tokens` and truncate RAG contexts aggressively.

Strategy 1: Model Tiering

Use the cheapest model that meets quality requirements:

```
def choose_model(task_complexity):
    """Route to appropriate model based on task complexity."""
    if task_complexity == "simple":
        # Greetings, FAQ, simple classification
        return "gpt-4o-mini"  # $0.15 / 1M input
    elif task_complexity == "medium":
        # Summarization, code explanation
        return "gpt-4o-mini"  # Still good enough
    else:
        # Complex reasoning, code generation, nuanced analysis
        return "gpt-4o"       # $2.50 / 1M input

# Example: simple tasks don't need the big model
model = choose_model("simple")
response = get_completion(messages, model=model)
```

> Rule of thumb: Start with mini. Upgrade only if quality is insufficient.

Strategy 2: Response Caching

Initialize an in-memory dictionary to store responses, along with hit/miss counters to measure cache effectiveness:

```
import hashlib
import json
import time

class ResponseCache:
    def __init__(self, ttl_seconds=3600):
        self.cache = {}
        self.ttl = ttl_seconds
        self.hits = 0
        self.misses = 0
```

Strategy 2: Response Caching

Implement retrieval and storage methods using hashed keys, ensuring stale entries are ignored based on the Time-To-Live (TTL):

```
    def get(self, query):
        key = hashlib.md5(query.lower().strip().encode()).hexdigest()
        entry = self.cache.get(key)
        if entry and time.time() - entry["timestamp"] < self.ttl:
            self.hits += 1
            return entry["response"]
        self.misses += 1
        return None

    def set(self, query, response):
        key = hashlib.md5(query.lower().strip().encode()).hexdigest()
        self.cache[key] = {"response": response, "timestamp": time.time()}
```

Strategy 3: Prompt Optimization

Reduce token count without losing quality:

```
# ❌ Verbose prompt (85 tokens)
"""You are an extremely helpful and knowledgeable assistant
who is an expert in Python programming. I would really
appreciate it if you could please help me understand what
the following error message means and provide a detailed
explanation of how to fix it."""

# ✅ Concise prompt (20 tokens)
"""You are a Python expert. Explain this error and how to fix it."""
```

Strategy 3: Prompt Optimization

Technique	Savings	Impact
Remove filler words	20-40%	None
Use `max_tokens` limit	Variable	May truncate
Shorter system prompts	10-30%	Test carefully
Compress RAG context	20-50%	May lose detail

Strategy 4: Truncate and Limit

Control response length to limit output token costs:

```
# Limit output tokens
response = get_completion(
    messages,
    max_tokens=150,       # cap output length
    temperature=0.0,      # deterministic = fewer retries
)
```

Strategy 4: Truncate and Limit

Truncate RAG context to save input tokens:

```
def limit_context(chunks, max_chars=2000):
    """Limit total context size for cost control."""
    context = ""
    used_chunks = []
    for chunk in chunks:
        if len(context) + len(chunk["text"]) > max_chars:
            break
        context += chunk["text"] + "\n\n"
        used_chunks.append(chunk)
    return used_chunks
```

Latency Optimization

Making Responses Feel Fast

KavinSchool

Latency Strategies

Strategy	How	Impact
Streaming	Show tokens as they arrive	Perceived latency drops 80%
Smaller models	GPT-4o-mini is 2-3x faster	Real latency reduction
Parallel calls	Run independent queries concurrently	Total time = max, not sum
Caching	Skip API call entirely	0ms for cached responses
Shorter prompts	Fewer tokens to process	Modest improvement

Streaming Responses

Show output as it arrives instead of waiting for the full response:

```
import litellm

def stream_response(messages):
    """Stream LLM response token by token."""
    response = litellm.completion(
        model="gpt-4o-mini", messages=messages, stream=True,
    )
    full_response = ""
    for chunk in response:
        delta = chunk.choices[0].delta.content or ""
        print(delta, end="", flush=True)
        full_response += delta

    print()  # newline after streaming
    return full_response
```

> Streaming doesn't reduce total latency, but it dramatically improves perceived latency.

Parallel API Calls

When you have multiple independent queries, run them concurrently:

```
import asyncio
import litellm

async def async_completion(messages, model="gpt-4o-mini"):
    """Async wrapper for LLM completion."""
    response = await litellm.acompletion(
        model=model, messages=messages
    )
    return response.choices[0].message.content
```

Parallel API Calls

Use `asyncio.gather` to execute independent LLM calls concurrently, significantly reducing total execution time:

```
async def parallel_queries(queries):
    """Run multiple queries in parallel."""
    tasks = [
        async_completion([{"role": "user", "content": q}])
        for q in queries
    ]
    return await asyncio.gather(*tasks)

# 3 queries: ~1 second total instead of ~3 seconds sequential
results = asyncio.run(parallel_queries([
    "Summarize Python lists",
    "Summarize Python dicts",
    "Summarize Python sets",
]))
```

Error Handling & Resilience

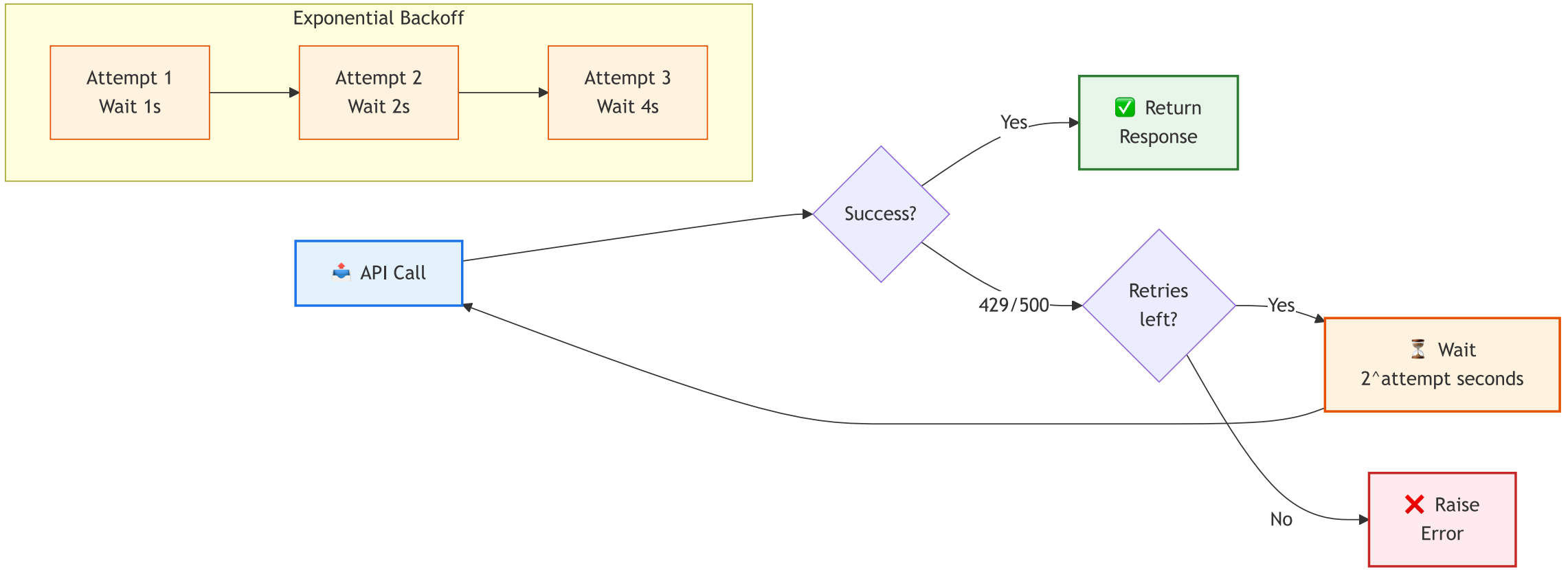
Building Reliable LLM Applications

KavinSchool

Common API Errors

Error	HTTP Code	Cause	Fix
Rate limit	429	Too many requests/minute	Retry with backoff
Server error	500	Provider outage	Retry, fallback provider
Timeout	408/504	Slow response	Increase timeout, use streaming
Auth error	401	Bad API key	Check key, rotate
Context too long	400	Input exceeds limit	Truncate, summarize
Content filtered	400	Safety filter triggered	Rephrase prompt

Retry with Exponential Backoff



- **Wait and Retry** - Gradually increase the delay (e.g., 1s, 2s) between retry attempts.
- **Targeted Recovery** - Only trigger retries for temporary failures like Rate Limits (429) or Server Errors (500)

Implementing Retry Logic

```python
import time
import litellm

def completion_with_retry(messages, max_retries=3, base_delay=1.0):
    """Call LLM with exponential backoff on retryable errors."""
    for attempt in range(max_retries + 1):
        try:
            response = litellm.completion(
                model="gpt-4o-mini", messages=messages,
            )
            return response.choices[0].message.content

        except litellm.exceptions.RateLimitError:
            if attempt == max_retries:
                raise
            delay = base_delay * (2 ** attempt)  # 1s, 2s, 4s
            print(f"Rate limited. Retrying in {delay}s...")
            time.sleep(delay)

        except litellm.exceptions.APIConnectionError:
            if attempt == max_retries:
                raise
            time.sleep(base_delay)

    raise RuntimeError("Max retries exceeded")
```

Monitoring API Usage

Initialize a tracker to record the token counts and latency metrics for each LLM API call you make:

```
import time

class UsageTracker:
    """Track API usage for monitoring and cost control."""

    def __init__(self):
        self.calls = []
        self.total_input_tokens = 0
        self.total_output_tokens = 0

    def log_call(self, model, input_tokens, output_tokens, latency_ms):
        self.calls.append({
            "model": model,
            "input_tokens": input_tokens,
            "output_tokens": output_tokens,
            "latency_ms": latency_ms,
            "timestamp": time.time(),
        })
```

Monitoring API Usage

Aggregate the tracked metrics to generate summaries, calculate averages, and monitor overall system health:

```
        self.total_input_tokens += input_tokens
        self.total_output_tokens += output_tokens
    def summary(self):
        return {
            "total_calls": len(self.calls),
            "total_input_tokens": self.total_input_tokens,
            "total_output_tokens": self.total_output_tokens,
            "avg_latency_ms": sum(c["latency_ms"] for c in self.calls)
                              / max(len(self.calls), 1),
        }
```

Spending Alerts

Establish a daily or monthly budget threshold to block requests if your accumulated spending exceeds the limit:

```
class BudgetGuard:
    """Prevent spending beyond a daily/monthly budget."""

    def __init__(self, daily_limit_usd=5.0):
        self.daily_limit = daily_limit_usd
        self.daily_spend = 0.0
        self.blocked_count = 0

    def check_budget(self, estimated_cost):
        """Return True if the call is within budget."""
        if self.daily_spend + estimated_cost > self.daily_limit:
            self.blocked_count += 1
            return False
        return True
```

Spending Alerts

Update the tracker with the actual cost of each successful call, and gracefully handle budget rejections in your application:

```
    def record_spend(self, actual_cost):
        self.daily_spend += actual_cost

# Usage
guard = BudgetGuard(daily_limit_usd=5.0)
if not guard.check_budget(estimated_cost=0.02):
    print("Budget exceeded! Using cached/fallback response.")
```

Session 2: Key Takeaways

1. **Output tokens cost 3-5x more.** Keep responses concise to save money
2. **Use the cheapest model that works.** GPT-4o-mini is great for simple tasks
3. **Cache aggressively.** Identical queries should never hit the API twice
4. **Stream for UX.** Users perceive streaming as much faster
5. **Retry with backoff.** Rate limits and errors are normal, not exceptional
6. **Monitor everything.** Track tokens, costs, latency per call

> Next: Session 3, Ethics, Bias Mitigation & Monitoring!

Questions?

Session 2 Complete. Break Time!

KavinSchool